

Nama : CALVIN WILLIAM WIJAYA
NIM : 10602008
Kelas : TRPL 2B
Mata Kuliah : SISTEM TERDISTRIBUSI

Soal 1:

Berdasarkan permasalahan yang dihadapi PT Sukses Bersama, arsitektur yang paling sesuai adalah arsitektur Microservices berbasis Cloud dengan Load Balancer dan Reverse Proxy. Pendekatan ini memungkinkan setiap fungsi bisnis berjalan sebagai layanan independen sehingga kegagalan satu komponen tidak meruntuhkan keseluruhan sistem.

a. Komponen – Komponen Utama Sistem

- API Gateway: pintu masuk tunggal yang menerima semua request dari client sebelum diteruskan ke service yang sesuai.
- Load Balancer: mendistribusikan traffic secara merata ke beberapa instance server agar tidak ada satu server yang kelebihan beban.
- Microservices: kumpulan layanan kecil yang masing-masing bertanggung jawab atas satu domain, misalnya Service Produk, Service Transaksi, Service Pengguna, dan Service Notifikasi.
- Message Broker (Apache Kafka / RabbitMQ): menangani komunikasi asinkron antar service sehingga request tidak saling menunggu.
- Database terdistribusi: setiap service memiliki database-nya sendiri untuk menghindari single point of failure pada lapisan data.
- Cache Layer (Redis): menyimpan data yang sering diakses di memori agar tidak perlu membaca database setiap saat.
- Monitoring & Logging (Prometheus + Grafana): memantau performa sistem secara real-time.

b. Peran Server dan Client

Server berperan sebagai penyedia layanan, menjalankan logika bisnis, menyimpan dan mengelola data, serta melayani permintaan dari client. Dalam arsitektur microservices, setiap service memiliki server-nya sendiri yang dapat di-scale secara independen. Sementara itu, client (aplikasi web maupun mobile yang digunakan pembeli dan penjual) hanya bertugas menampilkan antarmuka dan mengirimkan request ke API Gateway tanpa perlu mengetahui detail arsitektur internal.

c. Mekanisme Komunikasi Antar Komponen

- Komunikasi Sinkron (REST/gRPC): digunakan untuk operasi yang butuh respons langsung, misalnya ketika pengguna membuka halaman produk atau melakukan login.
- Komunikasi Asinkron (Message Queue via Kafka): digunakan untuk proses yang tidak perlu direspons seketika, seperti pengiriman notifikasi email setelah transaksi atau pembuatan laporan bulanan. Dengan cara ini, kegagalan pada satu service tidak akan memblokir service lainnya.
- Service Discovery: komponen seperti Consul atau Eureka memungkinkan setiap service menemukan alamat service lain secara otomatis tanpa konfigurasi manual.

d. Peningkatan Ketersediaan dan Performa

Arsitektur ini meningkatkan ketersediaan melalui beberapa mekanisme. Pertama, Load Balancer memastikan jika satu server penuh atau mati, traffic langsung dialihkan ke server lain tanpa gangguan bagi pengguna. Kedua, setiap microservice dapat direplikasi dalam beberapa instance sehingga satu service yang crash tidak menyebabkan seluruh platform down. Ketiga, Cache Layer mengurangi beban database secara drastis pada jam-jam sibuk karena data populer seperti daftar produk unggulan tidak perlu diambil dari database setiap saat.

e. Diagram Sederhana Arsitektur

Lapisan	Komponen	Fungsi
Client	Web App / Mobile App	Antarmuka pengguna (browser & smartphone)
Gateway	API Gateway + Load Balancer	Routing request dan distribusi traffic
Service	Microservices (Produk, Transaksi, User, Notif)	Logika bisnis masing-masing domain
Messaging	Apache Kafka	Komunikasi asinkron antar service
Cache	Redis	Penyimpanan data sementara berkecepatan tinggi
Storage	Database per Service (PostgreSQL / MongoDB)	Persistensi data terdistribusi
Monitoring	Prometheus + Grafana	Pemantauan performa dan alerting

Soal 2:

a. Mengapa HDFS lebih cocok daripada penyimpanan biasa?

Penyimpanan konvensional (satu server, satu harddisk) memiliki batas kapasitas dan tidak mampu menangani jutaan record transaksi secara efisien. HDFS (Hadoop Distributed File System) dirancang khusus untuk menyimpan data berukuran sangat besar dengan cara memecah file menjadi blok-blok kecil (umumnya 128 MB) lalu menyebarkannya ke banyak mesin fisik secara bersamaan. Keuntungannya antara lain: kapasitas dapat ditambah hanya dengan menambah server baru (horizontal scaling), proses baca-tulis bisa berjalan paralel sehingga jauh lebih cepat, dan data tetap aman meskipun ada mesin yang rusak karena setiap blok disimpan dalam beberapa salinan (replication).

b. Fungsi NameNode dan DataNode

Dalam ekosistem HDFS, ada dua peran utama:

- NameNode berperan sebagai 'peta' atau direktori utama. NameNode menyimpan metadata, yaitu informasi tentang di mana setiap blok data berada, nama file, izin akses, dan struktur direktori. NameNode tidak menyimpan data itu sendiri, melainkan hanya mencatat lokasinya. Analoginya seperti daftar isi sebuah buku besar kita tahu di halaman berapa setiap bab berada, tapi teks aslinya ada di halaman-halaman itu sendiri.
- DataNode berperan sebagai 'gudang' yang sebenarnya menyimpan blok-blok data. Setiap DataNode secara rutin mengirim laporan kondisi (heartbeat) ke NameNode agar

NameNode selalu tahu server mana yang masih aktif dan berapa banyak ruang yang tersedia.

c. Mekanisme Ketersediaan Data Saat Server Gagal

HDFS secara default menyimpan setiap blok data sebanyak 3 salinan (replication factor = 3) di DataNode yang berbeda. Misalkan PT Sukses Bersama memiliki data transaksi bulan Januari yang disimpan dalam Blok A. Blok A tersebut akan ada di DataNode-1, DataNode-2, dan DataNode-4 sekaligus. Jika DataNode-1 tiba-tiba mati, NameNode mendeteksinya melalui heartbeat yang tidak masuk, lalu secara otomatis memerintahkan DataNode lain untuk membuat salinan baru dari Blok A agar jumlah salinannya kembali ke 3. Dengan cara ini, pengguna tidak merasakan gangguan sama sekali.

d. Risiko Jika NameNode Mengalami Gangguan

NameNode adalah titik paling kritis dalam HDFS karena jika NameNode mati, tidak ada yang bisa membaca atau menulis data meskipun semua DataNode masih berjalan normal — karena tidak ada yang tahu di mana blok-blok data berada. Untuk mengatasi risiko ini, Hadoop menyediakan beberapa solusi:

- Secondary NameNode: bukan cadangan langsung (backup), melainkan bertugas secara periodik menggabungkan log perubahan metadata agar NameNode utama tidak kehabisan memori. Jika NameNode utama crash, Secondary NameNode bisa digunakan untuk memulihkan state terakhir.
- High Availability (HA) NameNode: solusi yang lebih kuat, yaitu menjalankan dua NameNode secara bersamaan (Active dan Standby). Jika NameNode aktif gagal, Standby NameNode langsung mengambil alih dalam hitungan detik tanpa downtime yang terasa oleh pengguna.

Soal 3:

a. Rekomendasi: Database Terdistribusi

Untuk kondisi PT Sukses Bersama yang memiliki data pelanggan tersebar di beberapa kota, database terdistribusi adalah pilihan yang jauh lebih tepat dibandingkan database terpusat. Alasan utamanya sederhana: jika semua data disimpan di satu lokasi terpusat (misalnya server di Jakarta), maka pengguna dari Surabaya atau Medan akan mengalami latensi tinggi karena setiap query harus melakukan perjalanan data yang jauh. Selain itu, server tunggal itu menjadi single point of failure jika rusak, seluruh layanan di semua kota akan terhenti.

b. Alasan Teknis

- Latensi rendah: dengan database yang ditempatkan lebih dekat ke pengguna (misalnya node di Jakarta, Surabaya, dan Medan), waktu respons menjadi jauh lebih cepat.
- Skalabilitas horizontal: saat pengguna bertambah, cukup tambah node baru tanpa harus mengganti server lama dengan yang lebih mahal.
- Fault tolerance: jika salah satu node gagal, node lain dapat terus melayani permintaan tanpa gangguan total.
- Paralelisme query: beban query dapat dibagi ke beberapa node sekaligus sehingga proses lebih cepat, terutama untuk laporan bulanan yang membutuhkan agregasi data besar.

c. Replikasi Data dan Partisi Data

Ada dua teknik utama yang digunakan dalam database terdistribusi:

- Replikasi: menyalin data yang sama ke beberapa node. Contohnya, data pelanggan premium bisa direplikasi ke semua node kota agar jika node Jakarta down, node Surabaya masih bisa melayani pengguna Jakarta yang sedang berbelanja. Replikasi meningkatkan ketersediaan dan kecepatan baca, namun membutuhkan sinkronisasi agar semua salinan tetap konsisten.
- Partisi (Sharding): membagi data ke beberapa node berdasarkan kriteria tertentu. Misalnya, data transaksi pelanggan dari Pulau Jawa disimpan di node Jakarta, sementara data dari luar Jawa disimpan di node Makassar. Teknik ini mengoptimalkan penggunaan penyimpanan dan kecepatan query karena setiap node hanya mengelola sebagian data.

d. Masalah Konsistensi Data dan Solusinya

Tantangan terbesar database terdistribusi adalah konsistensi, terutama saat terjadi pembaruan data secara bersamaan di beberapa node. Beberapa masalah yang umum terjadi:

- Stale Read: pengguna di Surabaya membaca data yang belum diperbarui karena sinkronisasi dari node Jakarta belum selesai. Solusi: gunakan strong consistency dengan protokol seperti Paxos atau Raft, atau terapkan eventual consistency dengan toleransi lag yang terukur.
- Split Brain: dua node menganggap dirinya masing-masing sebagai master dan menerima write secara bersamaan. Solusi: terapkan konsep quorum sebuah write dianggap berhasil hanya jika mayoritas node mengonfirmasi.
- Conflict Resolution: dua node memodifikasi data yang sama secara bersamaan. Solusi: gunakan vektor waktu (vector clock) atau teknik last-write-wins dengan timestamp yang terstandarisasi.

e. Contoh Situasi Nyata

Tokopedia dan Shopee, keduanya menghadapi tantangan serupa. Ketika promo flash sale berlangsung, jutaan pengguna dari seluruh Indonesia mengakses platform secara bersamaan. Mereka menggunakan pendekatan database terdistribusi dengan sharding berdasarkan ID pengguna dan replikasi data produk ke beberapa region. Hasilnya, meskipun ada lonjakan traffic hingga ratusan kali lipat, layanan tetap stabil karena beban terdistribusi secara merata.

Soal 4:

a. Mengapa Pemrosesan paralel diperlukan?

Setiap akhir bulan, PT Sukses Bersama harus memproses jutaan record transaksi dari seluruh Indonesia. Jika dikerjakan secara sekuensial oleh satu prosesor, prosesnya bisa memakan waktu berjam-jam atau bahkan berhari-hari. Pemrosesan paralel memecah pekerjaan besar itu menjadi banyak tugas kecil yang dikerjakan secara bersamaan oleh banyak mesin. Analoginya seperti menghitung suara pemilu: jauh lebih cepat jika ada ribuan petugas yang masing-masing menghitung satu TPS, dibandingkan satu orang yang menghitung semua TPS sendirian.

b. Konsep MapReduce

MapReduce adalah model pemrograman untuk memproses data besar secara terdistribusi yang terdiri dari dua fase utama:

- Map: setiap node membaca sebagian data dan menghasilkan pasangan key-value dari data tersebut. Pekerjaan dipecah dan dijalankan paralel di banyak mesin secara bersamaan.
- Reduce: hasil dari semua proses Map dikumpulkan, dikelompokkan berdasarkan key yang sama, lalu diagregasikan untuk menghasilkan output akhir.

Di antara keduanya, ada fase Shuffle & Sort yang secara otomatis memindahkan dan mengurutkan output Map ke node Reducer yang tepat.

c. Contoh Alur Kerja: Menghitung Jumlah Transaksi per Kota

Misalkan kita punya 6 record transaksi: Jakarta, Surabaya, Jakarta, Bandung, Surabaya, Jakarta.

Fase	Input	Output
Map	Jakarta, Surabaya, Jakarta, Bandung, Surabaya, Jakarta	(Jakarta,1), (Surabaya,1), (Jakarta,1), (Bandung,1), (Surabaya,1), (Jakarta,1)
Shuffle & Sort	Pasangan key-value dari semua Mapper	(Bandung,[1]), (Jakarta,[1,1,1]), (Surabaya,[1,1])
Reduce	(Bandung,[1]), (Jakarta,[1,1,1]), (Surabaya,[1,1])	(Bandung, 1), (Jakarta, 3), (Surabaya, 2)

d. Keuntungan dan Keterbatasan MapReduce

Keuntungan:

- Skalabilitas tinggi: menambah node baru langsung meningkatkan kapasitas pemrosesan tanpa perubahan kode.
- Toleransi kegagalan: jika satu node mati saat proses berjalan, Hadoop secara otomatis menjalankan ulang tugas itu di node lain.
- Cocok untuk batch processing data sangat besar yang tidak butuh hasil real-time.

Keterbatasan:

- Latensi tinggi: MapReduce tidak cocok untuk analisis real-time karena setiap siklus membaca dan menulis ke disk (HDFS) yang relatif lambat.
- Tidak efisien untuk data kecil: overhead setup cluster dan job scheduling bisa lebih lama daripada proses komputasi itu sendiri jika datanya sedikit.

- Model pemrograman kaku: tidak semua algoritma mudah diekspresikan dalam paradigma Map-Reduce. Untuk kasus seperti ini, framework modern seperti Apache Spark (yang memproses data di memori) sering menjadi alternatif yang lebih baik.

Soal 5:

Data transaksi yang diberikan:

No.	Produk
1	Laptop
2	Mouse
3	Laptop
4	Keyboard
5	Mouse
6	Laptop
7	Mouse
8	Monitor
9	Laptop
10	Keyboard

a. Proses Mapper Menghasilkan Pasangan (Key, Value)

Setiap record transaksi dibaca satu per satu, lalu Mapper menghasilkan pasangan (nama produk, 1) untuk setiap kemunculan:

No. Transaksi	Produk (Key)	Value
1	Laptop	1
2	Mouse	1
3	Laptop	1
4	Keyboard	1
5	Mouse	1
6	Laptop	1
7	Mouse	1
8	Monitor	1
9	Laptop	1
10	Keyboard	1

Output Mapper: (Laptop,1), (Mouse,1), (Laptop,1), (Keyboard,1), (Mouse,1), (Laptop,1), (Mouse,1), (Monitor,1), (Laptop,1), (Keyboard,1)

b. Proses Shuffle & Sort

Semua pasangan key-value dari Mapper dikelompokkan berdasarkan key yang sama, lalu diurutkan secara alfabetis:

Key (Produk)	Daftar Value
Keyboard	[1, 1]
Laptop	[1, 1, 1, 1]
Monitor	[1]
Mouse	[1, 1, 1]

c. Proses Reducer Total Penjualan Setiap Produk

Reducer menjumlahkan semua value dalam daftar untuk setiap key:

Produk	Proses Penjumlahan	Total Terjual
Keyboard	$1 + 1$	2
Laptop	$1 + 1 + 1 + 1$	4
Monitor	1	1
Mouse	$1 + 1 + 1$	3

d. Produk Paling Banyak Terjual

Berdasarkan hasil Reducer di atas, produk yang paling banyak terjual adalah Laptop dengan total 4 unit transaksi.

e. Alur Proses MapReduce Diagram

